

LTX-2.5 Local Video Generator

Install the open-source LTX-2.5 video model on your own machine
and generate multishot clips for free.

Why I stopped paying per clip

Every hosted AI video tool charges you to find out whether a prompt works. You pay for the clip where the hands go wrong, you pay for the one where the camera drifts, and you pay again for the take you actually use. On a short that needs four seconds of specific B-roll, that is three wasted charges to get one usable file.

LTX-2.5 is the open-source video model from Lightricks. You download the weights once, and after that every generation is free. The previous version passed 18 million downloads, so this is not a weekend research drop, it is a model a lot of people already run.

The promise in one line: one download, then unlimited clips on your own hardware, with multiple connected shots coming out of a single prompt instead of a fresh roll of the dice each time.

01

How it actually works

1. You download the pieces separately. LTX-2.5 is not one file. There is the transformer (the video model), a text encoder built on Gemma 4 12B that reads your prompt, a video VAE that turns the model's output into frames, an audio VAE because the model also makes sound, and two latent upscalers for resolution and frame rate. The full set is roughly 66 GiB.
2. You point a pipeline at those paths. The repo ships several pipelines as Python modules. You run one with a command, hand it the path to each weight file, add your prompt and a frame count, and it writes an MP4. Nothing phones home and nothing is metered.
3. Multishot happens inside one generation. Because the whole sequence is produced in a single pass, the model carries the same character, wardrobe and lighting from shot to shot. Separate generations have no memory of each other, which is why clips from most tools never quite match.

02

The one thing almost everyone gets wrong

People install the fast attention backend by default and then wonder why the install explodes on their Mac. The **natten** extra is Linux and CUDA only. On macOS and Windows the installer skips it and falls back to Triton or eager attention, which still runs the model, just slower.

WATCH OUT

If **uv sync --extra natten** fails on a Mac, run plain **uv sync** instead. You lose the fastest video VAE backend and nothing else. The second trap is the same shape: **--quantization fp8-scaled-mm** needs a Hopper or newer NVIDIA GPU. Use **--quantization fp8-cast** instead, which works with the bf16 checkpoints you already downloaded.

03

The files and commands

SOURCE CODE

The full repo, with every pipeline and the licence files:

<https://github.com/Lightricks/LTX-2>

1-INSTALL.SH

```
# 1. Get the code
git clone https://github.com/Lightricks/LTX-2.git
cd LTX-2

# 2. Install dependencies with uv
# 'natten' is the fastest attention backend for the video VAE,
# but it is Linux + CUDA only. On macOS and Windows the install
# skips it and falls back to Triton or eager attention.
uv sync --extra natten

# On a Mac, this is the safer line:
# uv sync
```

2-DOWNLOAD-WEIGHTS.SH

```
# Log in to Hugging Face first.
# The LTX-2.5 weights are gated: accept the model terms on the
# Lightricks/LTX-2.5 page, and use a READ token with the
# "read gated repos" scope switched on.
hf auth login

# Download the five files the fast pipeline needs into models/ltx-2.5
hf download Lightricks/LTX-2.5 \
  diffusion_models/ltx-2.5-22b-distilled-transformer-bf16.safetensors \
  text_encoders/gemma4-12b-with-proj-ltx-2.5-bf16.safetensors \
  vae/ltx-2.5-video-vae-bf16.safetensors \
  vae/ltx-2.5-audio-vae-bf16.safetensors \
  latent_upscale_models/ltx-2.5-latent-spatial-upscaler-x2-bf16-1.0.safetensors \
  --local-dir models/ltx-2.5

# Optional, only for the higher quality DFR pipeline:
hf download Lightricks/LTX-2.5-22b-IC-LoRA-Pixel-Spatial-Upscaler \
  ltx-2.5-22b-ic-lora-pixel-spatial-upscaler-x2-1.0.safetensors \
  --local-dir models/ltx-2.5/loras
```

3-GENERATE.SH

```
# The fast pipeline. This is the one to test with.
# --num-frames 121 is about five seconds at 24 fps.
uv run python -m ltx_pipelines.distilled \
    --transformer-path models/ltx-2.5/diffusion_models/ltx-2.5-22b-distilled-
transformer-bf16.safetensors \
    --text-encoder-path models/ltx-2.5/text_encoders/gemma4-12b-with-proj-ltx-2.5-
bf16.safetensors \
    --video-vae-path models/ltx-2.5/vae/ltx-2.5-video-vae-bf16.safetensors \
    --audio-vae-path models/ltx-2.5/vae/ltx-2.5-audio-vae-bf16.safetensors \
    --spatial-upscaler-path models/ltx-2.5/latent_upscale_models/ltx-2.5-latent-
spatial-upscaler-x2-bf16-1.0.safetensors \
    --num-frames 121 \
    --seed 42 \
    --output-path output.mp4 \
    --prompt "YOUR_PROMPT_HERE"
```

4-PRODUCTION-QUALITY.SH

```
# The production pipeline. Slower, more memory, sharper output.
# It adds the detailing LoRA you downloaded as the optional step.
uv run python -m ltx_pipelines.dfr_pipeline \
    --transformer-path models/ltx-2.5/diffusion_models/ltx-2.5-22b-distilled-
transformer-bf16.safetensors \
    --text-encoder-path models/ltx-2.5/text_encoders/gemma4-12b-with-proj-ltx-2.5-
bf16.safetensors \
    --video-vae-path models/ltx-2.5/vae/ltx-2.5-video-vae-bf16.safetensors \
    --audio-vae-path models/ltx-2.5/vae/ltx-2.5-audio-vae-bf16.safetensors \
    --detailing-lora models/ltx-2.5/loras/ltx-2.5-22b-ic-lora-pixel-spatial-upscaler-
x2-1.0.safetensors \
    --spatial-upscaler-path models/ltx-2.5/latent_upscale_models/ltx-2.5-latent-
spatial-upscaler-x2-bf16-1.0.safetensors \
    --num-frames 121 \
    --seed 42 \
    --output-path output_dfr.mp4 \
    --prompt "YOUR_PROMPT_HERE"
```

FLAGS-CHEATSHEET.TXT

```
# Duration. Frames, not seconds. 121 frames is roughly 5s at 24 fps.
--num-frames 121

# Resolution. Defaults to 1024x1536. UHD 4K is 3840x2176.
--width 1024 --height 1536

# Frame rate multiplier. 0, 1 or 2. Each step doubles the fps.
--temporal-upscalings 1

# Reproducibility. Same seed + same prompt = same clip.
--seed 42

# Memory. Use this first when a generation runs out of VRAM.
--quantization fp8-cast

# Memory, harder. Pushes model parts off the GPU. Costs speed.
--offload cpu
--offload disk
```

THE MULTISHOT PROMPT SHAPE

Shot 1: [SUBJECT] [DOING WHAT] in [LOCATION], [CAMERA ANGLE],
[LIGHTING DESCRIPTION].

Shot 2: the same [SUBJECT], now [SECOND ACTION], [NEW CAMERA ANGLE],
same [LIGHTING DESCRIPTION].

Shot 3: the same [SUBJECT], [THIRD ACTION], [THIRD CAMERA ANGLE],
same [LIGHTING DESCRIPTION].

Keep the same [SUBJECT] wardrobe, hair and face across all three shots.
Keep the light direction and colour temperature identical throughout.

SITES SHOWN IN THE VIDEO

- Lightricks/LTX-2 on GitHub
<https://github.com/Lightricks/LTX-2>

Set it up in Claude Code or Codex

1. Pick a folder with room to spare. You need about 66 GiB free for the weights plus space for output files. Put it somewhere with a real drive, not a network share.
2. Clone the repo and open it. Run **git clone <https://github.com/Lightricks/LTX-2.git>**, then open that folder in Claude Code or Codex so the agent can see the README and the pipeline modules.
3. Let the agent read the README first. Ask it to read **README.md** and tell you which pipeline matches what you want. The repo ships a fast one, a production one, an image-to-video one, a keyframe interpolation one, an audio-to-video one, one that regenerates a time region of a clip you already have, and one for dubbing.
4. Install dependencies. On Linux with an NVIDIA card, **uv sync --extra natten**. On a Mac or Windows, **uv sync**. If **uv** is not on the machine, ask the agent to install it before anything else.
5. Set up your Hugging Face token. Accept the model terms on the Lightricks/LTX-2.5 page while logged in, then create a read token with the "read gated repos" scope enabled. Ask the agent to add it to your shell profile as **HF_TOKEN**, then **open a new terminal window** so the variable is actually loaded. Run **hf auth login** to confirm.
6. Download the weights. Paste the **hf download** command from 2-download-weights.sh. This is the long part. On a normal connection expect it to run for a while, and check the **models/ltx-2.5** folder afterwards to confirm the subfolders landed.
7. Run the first test. Use 3-generate.sh with a short, plain prompt and **--num-frames 121**. Keep **--seed 42** so you can reproduce it. Do not change resolution yet.
8. Check what success looks like. You get an **output.mp4** in the folder, about five seconds long, with audio on it. If the file exists and plays, the install is correct and everything after this is prompt work rather than setup work.

QUICK TIPS

- Keep the weights outside the repo folder if you plan to pull updates. A fresh clone should not mean a fresh 66 GiB.
- Write your working command into a shell script the first time it succeeds. Those paths are long and retyping them is where typos come from.
- Test with the convolutional video VAE (**ltx-2.5-video-vae-conv-bf16.safetensors**) if the diffusion decoder gives you dependency trouble. It is lighter and needs no extra packages.
- Same seed plus same prompt gives you the same clip, so change one thing at a time when you are tuning a shot.

04

The prompts I actually use

GET IT INSTALLED WITHOUT READING THE README YOURSELF

Read the README.md in this repo. Then install LTX-2.5 on this machine. I am on [MACOS / LINUX WITH NVIDIA / WINDOWS]. Tell me before you run anything that downloads more than a gigabyte, and skip the natten extra if my platform does not support it. Stop when the dependencies are installed and tell me what the next command is.

DOWNLOAD ONLY THE WEIGHTS I NEED

Using the download commands in the README, fetch only the weights required for the distilled pipeline into models/ltx-2.5. Do not download the dev transformer or the DFR LoRA. My Hugging Face token is in HF_TOKEN. After it finishes, list what landed and the size of each file.

TURN A SHOT LIST INTO ONE MULTISHOT PROMPT

Here is my shot list for a [LENGTH]-second clip about [TOPIC]:

[PASTE SHOT LIST]

Rewrite it as a single LTX-2.5 prompt that generates all the shots in one pass. Describe each shot in order, and explicitly repeat the subject's wardrobe, hair and the lighting direction in every shot so the model holds them. Keep it under [WORD COUNT] words.

BUILD ME A REPEATABLE GENERATE SCRIPT

Write a shell script called gen.sh that wraps the ltx_pipelines.distilled command with all five weight paths filled in. It should take the prompt as the first argument, the output filename as the second, and default to --num-frames 121 and --seed 42. Make seed and frame count overridable with environment variables.

MAKE IT FIT IN LESS MEMORY

My last generation failed with an out of memory error on a [GPU OR MAC MODEL] with [AMOUNT] of memory. Modify my generate command to use the least aggressive memory setting that will fit, starting with --quantization fp8-cast before you reach for --offload. Explain what each change costs me in speed or quality.

When it breaks

ERROR	FIX
401 or 403 on hf download	The repo is gated. Accept the model terms on the Lightricks/LTX-2.5 Hugging Face page while logged in, then use a read token with the "read gated repos" scope enabled.
natten fails to build or install	It is Linux and CUDA only. Run plain <code>uv sync` with no <code>--extra natten`</code>. The video VAE falls back to Triton or eager attention and still works.</code>
Out of memory during generation	Add <code>--quantization fp8-cast`</code> first. If it still will not fit, add <code>--offload cpu`</code> , then <code>--offload disk`</code> as a last resort. Both trade speed for finishing.
fp8-scaled-mm errors out	That quantization mode needs a Hopper or newer NVIDIA GPU. Use <code>--quantization fp8-cast`</code> instead.
File not found on a --*-path flag	Every pipeline flag wants the full path to a specific .safetensors file, not a folder. Check that <code>hf download`</code> created the <code>diffusion_models`</code> , <code>text_encoders`</code> , <code>vae`</code> and <code>latent_upscale_models`</code> subfolders under <code>models/ltx-2.5`</code> .
Video VAE wants dependencies you cannot install	Download <code>ltx-2.5-video-vae-conv-bf16.safetensors`</code> and point <code>--video-vae-path`</code> at it. The convolutional decoder is lighter and needs no extra packages.
Clip is the wrong length	Duration is set in frames, not seconds. <code>--num-frames 121`</code> is about five seconds at 24 fps, so scale from there.
Output looks low resolution or choppy	Defaults are 1024x1536. Set <code>--width`</code> and <code>--height`</code> for size, and <code>--temporal-upscalings 1`</code> or <code>2`</code> to double or quadruple the frame rate.

Ready to go deeper?

This playbook is the short version. Inside the hub you get the full build walkthroughs, the prompt library, and the systems I use to ship this in a weekend.

Join the Build →

<https://hub.digicuratoragency.com/join>